Lab 6 Report: Path Planning Using Monte Carlo Localization, Probabilistic Road Maps, and Pure Pursuit

Team 2

Nina Garcia-Ruiz
Brendon Murphy
Irene Tang
Vandell Vatel

6.4210 Robotics: Science and Systems

April 10, 2026

1 Introduction

Autonomous mobile robots require reliable knowledge of their own position within a known environment - a problem known as *localization* - as well as a the ability to safely navigate through these environments to a destination, known as *path planning*. The system uses two pieces of sensor data: wheel odometry, which tracks how the car has moved over time, and LiDAR scan data, which measures the distances to surrounding obstacles. Together, these allow the robot to maintain a belief state over its pose (position and orientation) and to calculate the optimal path to get to a destination.

This lab extends our localization model in Lab 5, improves upon it, and introduces our path planning model. Our localization model consists of three modules - a motion model, a sensor model, and a particle filter, while our path planner consists of two - a path finder and a path follower - all implemented in Python using ROS2. We evaluated the system in both a simulator and on the physical car in the MIT Stata basement.

2 Localization Technical Approach

Monte-Carlo Localization allows us to track our pose using a distribution. We maintain a set of N particles $\mathbf{x} = \{x^{(1)}, \ldots, x^{(N)}\}$, where each particle

$x^{(i)} = [x, y, \theta]^\top$ represents a candidate pose of the robot from that distribution. Localization is then a process of updating that distribution via the particles. At each timestep the filter performs two steps to update the particles: (1) it propagates particles forward using the motion model given new odometry data, (2) it reweighs particles and assigns each one likelihoods given new LiDAR scans. Particles are then resampled proportional to their weights to maintain a fixed number of N particles. Finally, the robot's estimated pose is take as the circular mean of the particle distribution.

2.1 Motion Model

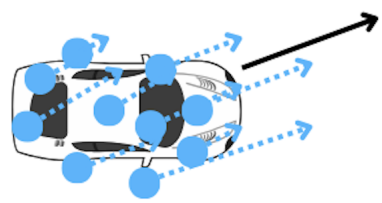

Figure 1: Diagram visualizing the motion model. The blue dots represent the particles. The black arrow represents the odometry. The blue dashed arrows represent the odometry updating each particle.

The motion model updates previous particle poses $\mathbf{x}_{k-1}$ using noisy odometry data $\mathbf{u}_k = [\Delta x, \Delta y, \Delta\theta]^\top$ to predict the robot's new state $\mathbf{x}_k$. By simulating the robot's kinematics, particles track the car's physical trajectory:

$$\mathbf{x}_k = f(\mathbf{x}_{k-1}, \mathbf{u}_k + \epsilon_u) \tag{1}$$

where f defines the deterministic kinematics and ϵ_u represents odometry noise.

Update Step

To apply the deterministic update f, we separate the pose and odometry into position and orientation components. We transform the local odometry position into the global frame using the rotation matrix $\mathbf{R}(\theta_{k-1})$ and add the orientation change directly:

$$\mathbf{x}_k = f(\mathbf{x}_{k-1}, \mathbf{u}_k) = \begin{bmatrix} \mathbf{x}_{k-1,\text{pos}} + \mathbf{R}(\theta_{k-1})\mathbf{u}_{k,\text{pos}} \\ \mathbf{x}_{k-1,\theta} + \mathbf{u}_{k,\theta} \end{bmatrix} \tag{2}$$

where $\mathbf{x}_{k,\text{pos}} = \begin{bmatrix} x_k \\ y_k \end{bmatrix}$, $\mathbf{u}_{k,\text{pos}} = \begin{bmatrix} \Delta x \\ \Delta y \end{bmatrix}$ $\mathbf{x}_{k,\theta} = \theta_k$, and $\mathbf{u}_{k,\theta} = \Delta\theta$.

Noising Step

Real-world kinematics suffer from cross-coupled errors in translation and rotation. for instance, rotating causes translational slip, and translating causes rotational drift. To model this, we sample noise $\epsilon_u = [\epsilon_x, \epsilon_y, \epsilon_\theta]^\top$ from zero-mean Gaussians: $\mathcal{N}(0, \sigma_x^2)$, $\mathcal{N}(0, \sigma_y^2)$, and $\mathcal{N}(0, \sigma_\theta^2)$.

These standard deviations scale based on the variance of the odometry data and user-defined α coefficients:

$$\sigma_x = \sqrt{\alpha_1(\Delta x)^2 + \alpha_2(\Delta\theta)^2}$$
$$\sigma_y = \sqrt{\alpha_1(\Delta y)^2 + \alpha_2(\Delta\theta)^2}$$
$$\sigma_\theta = \sqrt{\alpha_3(\Delta x)^2 + \alpha_4(\Delta\theta)^2}$$

Here, α_1 and α_4 scale independent noise (effect of position and orientation change on translational and rotational noise respectively), while α_2 and α_3 govern the cross-coupled noise (effect of position change on rotational noise and effect of orientation change on translational noise).

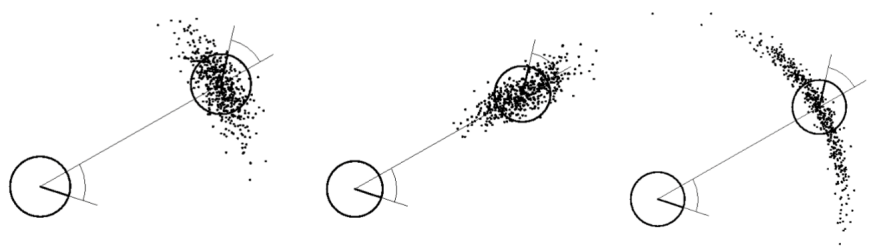

Figure 2: Representations of how the noise term affects the particles' spread given the robot's odometry. The first example occurs when there is similar position and orientation change, the second example occurs when there is large position change compared to orientation, and the third example occurs when there is large orientation change compared to position.

Optimizations

We implemented two modifications to improve real-world performance:

- **Midpoint Integration:** Instead of rotating $\mathbf{u}_k$ by the initial heading θ_{k-1}, we use the average heading $\theta_{\text{avg}} = \theta_{k-1} + \frac{\Delta\theta}{2}$. This better approximates continuous curved motion rather than instantaneous point-turns at each time step.

- **Forward Bias:** To counteract system latency that causes particles to artificially lag behind the ground-truth car, we scale Δx by a multiplier $\alpha_{\text{forward}} > 1.0$, pushing the particle cloud slightly ahead to better match where they should be.

2.2 Sensor Model

The sensor model assigns a likelihood to each particle by comparing the LiDAR scan the robot actually observes to the scan it would have hypothetically observed at each particle's position, computed by ray-casting. Ray-casting works by shooting beams from each particle and calculating the distance to the nearest obstacles.

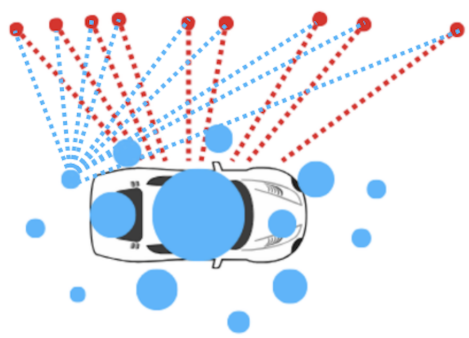

Figure 3: Each blue dot represents a particle, each blue line represents a ray-cast from the particle, and each red line represents a LiDAR beam. We compare the length of each blue line to the length of each red line to compute the likelihood of the blue dot occurring.

Precomputed Lookup Table

To achieve efficient real-time performance with many particles and beams, we precompute a 201×201 lookup table $p(z \mid d)$, where z is the discretized observed range (in pixels) and d is the discretized ray-cast range that we assume to be the ground-truth distance. This table is computed once at startup and indexed at runtime.

The probability of a single beam measurement is a weighted sum of four com-

ponents:

$$
\begin{aligned}
p(z \mid d) = \\
& \alpha_{\text{hit}}\, p_{\text{hit}}(z \mid d) \\
& + \alpha_{\text{short}}\, p_{\text{short}}(z \mid d) \\
& + \alpha_{\text{max}}\, p_{\text{max}}(z \mid d) \\
& + \alpha_{\text{rand}}\, p_{\text{rand}}(z \mid d) \\
& \text{s.t. } \alpha_{\text{hit}} + \alpha_{\text{short}} + \alpha_{\text{max}} + \alpha_{\text{rand}} = 1
\end{aligned} \tag{3}
$$

Each of the four probability models ($p_{hit}, p_{short}, p_{max}$, and p_{rand}), represent distinct physical phenomena and allow us to encode properties about the environment that the LiDAR detects:

1. **Hit** (p_{hit}): the probability that the beam strikes an expected map obstacle, producing a reading close to d with Gaussian noise $\sigma_{\text{hit}} = 10.0$ pixels. This is the dominant term: $\alpha_{\text{hit}} = 0.80$.

2. **Short** (p_{short}): the probability of an unexpected obstacle (e.g., a person or clutter) that causes a shorter-than-expected reading. Modeled as a linearly decreasing distribution over $[0, d]$: $\alpha_{\text{short}} = 0.07$.

3. **Max** (p_{max}): the probability of a large reading due to a highly reflective or transparent surface, modeled as a point mass at z_{max}: $\alpha_{\text{max}} = 0.07$.

4. **Random** (p_{rand}): the probability of complete randomness, modeled by a uniform distribution over $[0, 201]$: $\alpha_{\text{rand}} = 0.06$.

After computing the cumulative probability, or the product of each four probability models, each column of the table is normalized to sum to 1 (we want probability over each z_k to sum to 1).

Particle Weight Computation

At runtime, the sensor model recieves LiDAR scans and the current particle set. For each particle, it uses PyScanSimulator2D to ray-cast the expected scan for that particle's pose on the map. The observed ranges and the ray-cast ranges are converted from meters to pixels using the formula

$$
n_{pixels} = \frac{n_{meters}}{\text{map resolution} \times \text{lidar to map scale}}
$$

and the updated pair z_k, d is used for look up in the precomputed table detailed in section **2.3.1**.

If we assume that each ray-cast beam is independent, then the joint probability of the full scan per particle is the product of each beam's probabilities:

$$p(z_k \mid x_k, m) = \prod_{i=1}^{n} p\Big(z_k^{(i)} \mid x_k, m\Big) \tag{4}$$

To prevent numerical underflow from multiplying many small probabilities and to reduce the sensitivity to outlier beams, we raise each particle's weight to the power 0.7 (i.e., we flatten the weight distribution):

$$w^{(i)} \leftarrow \Big(w^{(i)}\Big)^{0.7} \tag{5}$$

2.3 Particle Filter

Initialization

When a pose estimate is sent via RViz's "Pose Estimate" tool (publishing to the topic `/initialpose`), the particle filter initializes all N particles at that pose with small Gaussian perturbations ($\sigma = 0.5$ m, 0.5 rad).

Resampling

We use low-variance sampling (also known as systematic resampling), which draws a single random number $r \sim \text{Uniform}(0, 1/N)$ and places N evenly spaced pointers:

$$c_j = r + \frac{j-1}{N}, \quad j = 1, \ldots, N \tag{6}$$

Each pointer selects a particle from the cumulative weight distribution circularly. We used low-variance sampling as opposed to multinomial resampling, since this method reduces sampling variances and produces better particle diversity (prevents collapse).

After resampling, a small amount of Gaussian noise ($\sigma = 0.05$ m for position, $\sigma = 0.02$ rad for heading) is added to each resampled particle to model an appropriate spread with sensor noise.

Average Pose Estimate

To calculate our average pose estimate from our particle cloud, we take the average of the X and Y coordinates and a circular mean of the orientation angle. This average was chosen to take into account the circular nature of angles. We believe it is representative enough of the coordinates under the assumption that we are resampling from the weighted distribution.

3 Path Planning Technical Approach

With localization in place, we implemented a full path planning and trajectory following pipeline to enable the car to autonomously navigate from a start to end pose in the Stata basement. Our approach has two modules: (1) path construction using Probabilistic Road maps (PRMs) and Dijkstra's search and (2) path following with pure pursuit.

3.1 Path Finding

We start with an occupancy grid that encodes obstacles as occupied cells. Before trajectory building, we inflate every obstacle by a safety radius

$$r_{\text{safety}} = 0.60\text{m}$$

using `cv2.distanceTransform`. Cells that have a distance to the nearest obstacle less than r_{safety} are marked unsafe and thus, another obstacle. Planning on the inflated map allows us to reduce the car to a point representation on the map while guaranteeing no physical collisions.

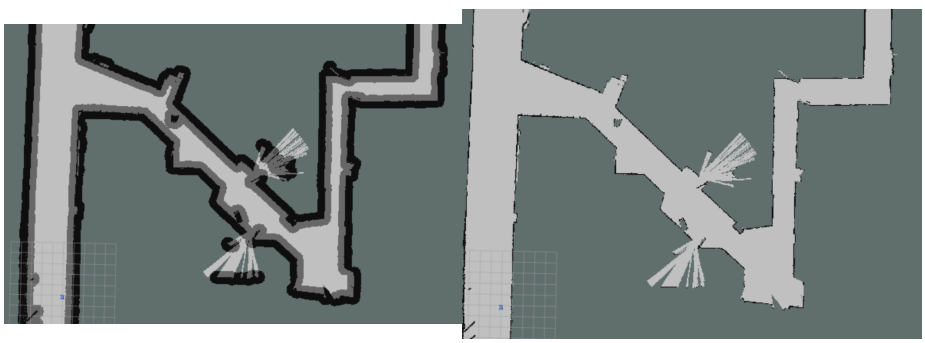

Figure 4: The inflated grid and the regular grid side-by-side.

A probabilistic road map is built once offline (when the car is not running) and saved as a `networkx.Graph`, together with a `scipy.spatial.KDTree` for efficient nearest-neighbor lookup.[1] In pseudo-code, our PRM construction follows this structure:

[1] A KD-tree, in brief, recursively partitions the map along different axes. This splits 2D space into hyper-rectangles, enabling efficient nearest-neighbor querying.

```
Input: occupancy map, number of samples n, connection radius r
Output: graph G, KD-tree T
Sample n collision-free nodes uniformly at random over free cells in the
 occupancy map. Build G and T simultaneously; for each node u do
    Query T for all neighbors within radius r; for each neighbor v (with
     v ≠ u) do
        if line segment uv is collision-free then
            Add edge (u, v) with weight |u − v|;
        end
    end
end
return G, T
```

Algorithm 1: Offline PRM Graph and Tree Construction

Collision checking for each line segment is performed by sampling the segment at intervals of 0.5 pixels and verifying that every interpolated grid cell is unoccupied. When the PRM is loaded for online planning, G represents feasible positions in the map and their connectivity, while T enables efficient connection of start and goal positions to the graph.

3.1.1 Online Path-Finding

When a start pose and goal pose are received on the car, the points are temporarily inserted into the static PRM graph as a special nodes `"start"` and `"end"`. The KD-tree is queried for all existing nodes within the same connection radius of 5.0 and collision-free edges are added.

We then run Dijkstra's with edge weights being the Euclidean distance between the connected nodes. After planning, the temporary start and end nodes are removed to allow the graph to be queried again.

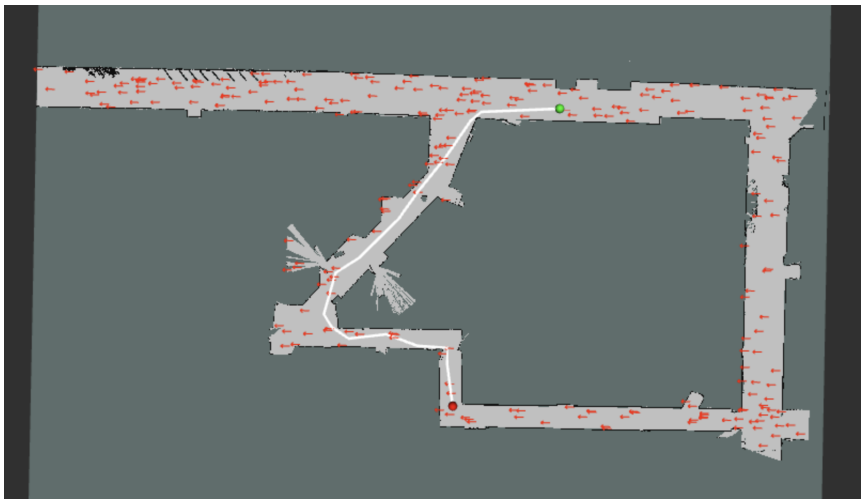

Figure 5: Using Dijkstra's in simulation shows the algorithm tracing a distance-optimal path through the graph.

3.2 Grid-Search Based Path Finding

Another approach we considered for path finding (*and ultimately chose not to use*) was to systematically explore all unoccupied cells in the occupancy grid, rather than exploring a smaller connected graph of randomly sampled points.

3.2.1 Method

Following the previous approach, we inflate the occupancy grid by a designated safety radius so that we can prevent collisions as we explore. However, instead of using Dijkstra's algorithm to find a path to the goal, we use A*, a more general search algorithm. Using A*, we can assign a cost to each explored path A^*_{cost} as a weighted sum of **g**, the current true cost of the path (often just the total path distance) and **h**, a heuristic function that estimates the future cost to reach the goal.

$$A^*_{\text{cost}}(path) = w_{path} * g(path) + w_{goal} * h(path)$$

In the special case where $w_{path} \geq w_{goal}$ and the heuristic is always a underestimate of the true future cost to reach the goal, A^* is guaranteed to return the optimal path. Initially, we decided to maintain this optimality by choosing the Euclidean distance to the goal as our heuristic as it assumes an optimistic, straight-line path to the goal thus underestimating the cost of the true path. In simulation, while this approach was able to find optimal paths, the computation time increased sharply with slight increases in the initial distance to the goal, a problem we attributed to Single-Step Exploration.

3.2.2 Single-Step vs. Multi-Step Exploration

Single-Step Exploration guaranties optimality by exploring every possible cell at the cost of greatly increasing the computation time. A* works by constantly choosing to further explore the path with the lowest cost, but when steps are taken one at a time, not only are there more paths to consider, but it is harder to narrow down quickly on the optimal path since closely neighbored cells will have similar costs because they are at similar distances from the goal. Multi-Step Exploration basically involves skipping cells by exploring with a step size greater than 1. By taking larger steps, we drastically reduce the number of cells we actually explore, and, furthermore, a larger step towards the goal is more differentiable from a larger step away from it, allowing A* to run more efficiently. Since we are not exploring every cell, Multi-Step Exploration does lose the optimality guarantee. In simulation, however, we found that it only resulted in a slight increase compared to the optimal cost returned by Single-Step Exploration, which is further explained in our Experimental Evaluation.

3.3 Path Following

Once we have a trajectory, we are able to follow it with an implementation of pure pursuit. First, we the first point included in the trajectory that is at least a certain lookahead distance away from the current pose estimate of the car. Then we use the pure pursuit steering law to navigate towards it. This means our driving commands are highly dependent on the specific points included in the trajectory.

4 Experimental Evaluation

First, we will isolate localization results, then path planning methods, and lastly combine all for path following analysis.

4.1 Localization Results

4.1.1 Simulation

We find that our particle filter is very accurate in simulation. Accuracy is measured using CTE, which yields low values between 0.2-0.4 meters for position and near zero values for orientation. We observed these low CTE values in the main hallway, bike hallway, and full loop around the Stata basement, showing that in simulation our particle filter is robust to changes in environment.

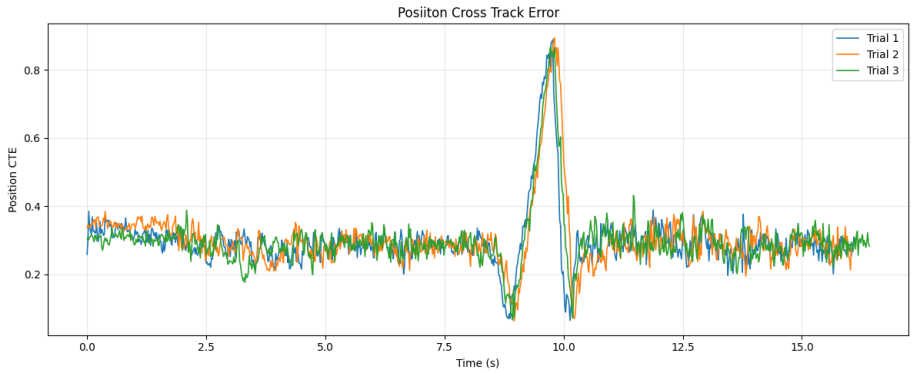

Figure 6: Cross-track error for position and orientation in simulation for the main hallway. The large spike in cross-track error occurs when there is a brief bend in the path with obstacles, causing uncertainty in the sensor model about the robot's current position. However, the graph shows that position error hovers between 0.2-0.4 meters and orientation error is almost negligible.

4.1.2 Real-World

Real-world Trial 1 (Main Hallway of Stata). Qualitative analysis of the localizer shows that it performs well in the Main Hallway of the Stata Basement. This hallway has many unique features that make it easier to localize in this section of the map. Additionally, conditions in the real world during the runs matched the map well which also contributes to good localization.

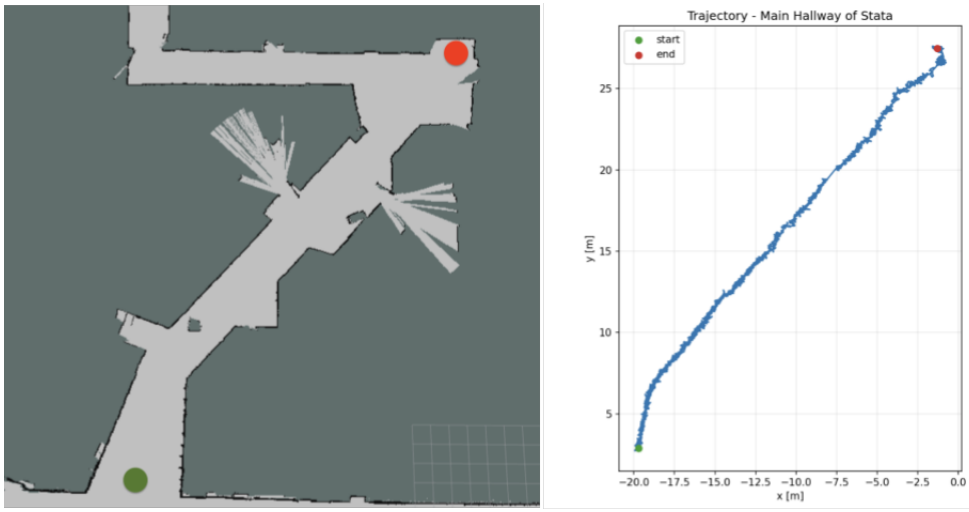

Figure 7: Plot of localized poses is relatively smooth and accurately matches the path taken through the map in the real world (based on comparison to video of the run).

Real-world Trial 2 (Bike Hallway).
When testing localization on another sector of the map, the resulting estimated path was not nearly as smooth. Indeed, localization performed poorly in hallways with clutter.

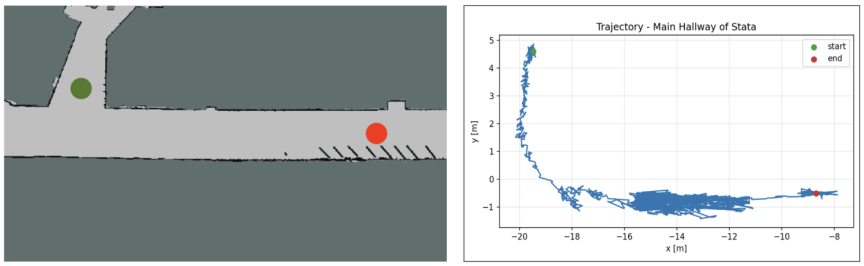

Figure 8: The first iteration of particle filtering model. Stutter can be observed from the interval $x \in [-16, -11]$.

Noise and Parameter Tuning to Prevent Weight Collapse
A common issue in localization is a mismatch between the real-world and assumed map layout. Unexpected clutter can cause particle/weight collapse, where the sensor model resamples only a few likely particles, and the rest ef-

fectively vanish due to extremely low probabilities even if they are closer to the ground truth pose.

To address this, we flattened weights in the sensor model. Flattening the sensor model weights reduces "peaks" in the weight distribution, so resampling draws from a more diverse set of points to avoid particle collapse.

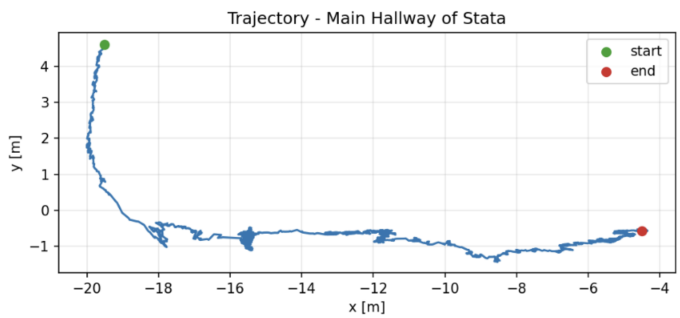

Figure 9: Second iteration: weight flattening and added noise together push the car past the original failure point.

As a final adjustment, we reduced the frequency at which we use the sensor model to collapse the particles to 1 Hz. Relying more on the motion model gives us more accurate and physically feasible trajectories, even with the noise added in the motion model since it is zero mean and wouldn't affect the average pose estimate. Notably, the estimated endpoint in Figure 10 is closer to the ground truth and smoother.

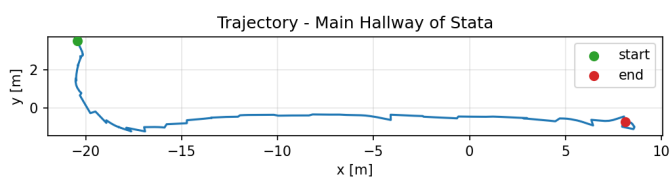

Figure 10: Third iteration: Updating with sensor model at a lower frequency leads to more physically feasible trajectories while being accurate to ground truth.

4.1.3 Computation Time

Similarly to how it is important that our map match the existing world for good localization, it is also important that our perception of the world be relevant to where we currently are in the world. Updating the particles via the sensor model in a timely matter is an important way of maintaining this accuracy.

Across all of our different runs, the computation time of the sensor model was consistently under 60 microseconds. This is a rate of nearly 17 kHz. The LiDAR scans were published at a rate of 85 Hz. Relative to the rate at which scans are received, the time it takes to do the computation on them is sufficiently fast and gives us accurately updated particles.

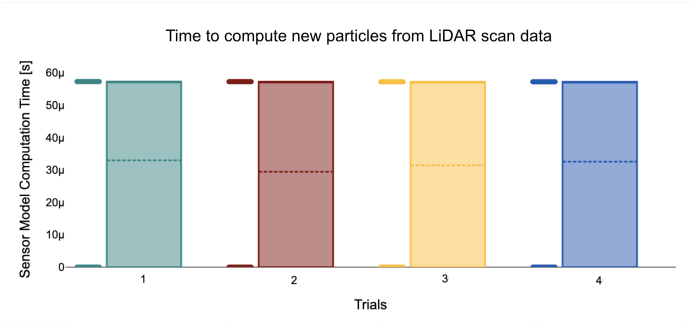

Figure 11: Consistently under 60 microseconds to compute new particles from scan data across multiple trials.

4.2 Path Planning Results

4.2.1 Simulation Evaluation

We compared sampling-based (PRM) and grid-based planning, focusing on tradeoffs between computation time, path cost, and safety.

Sampling-Based Planning: Effect of Graph Size (n)

We evaluated PRM performance across varying n, measuring path cost (the total Euclidean distance along the trajectory) and computation.

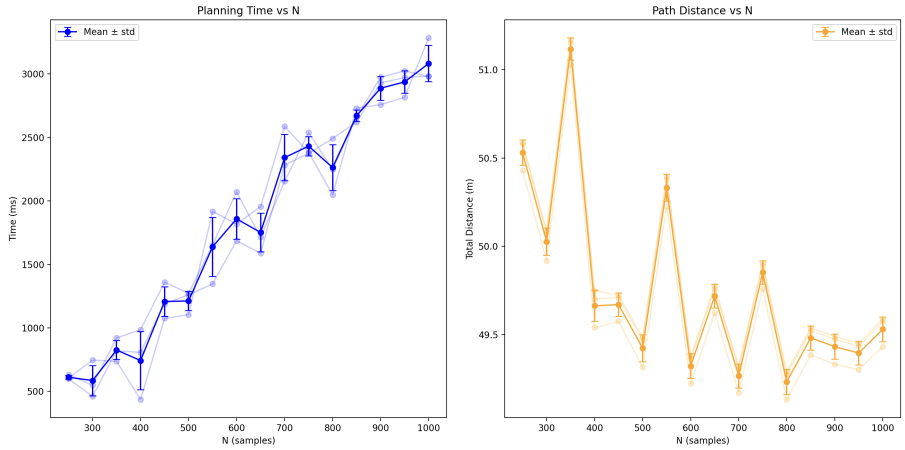

Figure 12: Planning time and path distance vs. graph size n.

From the graph above, as n increases, computation time grows while path cost decreases (improved optimality), signifying an important tradeoff between efficiency and optimality. Additionally, we observed that more optimal paths hug walls and take sharper corners, raising safety concerns.

To quantify the safety of the path, we calculate the clearance of the path. The clearance is defined as the distance to the nearest obstacle along the path, or:

$$d_{\text{min,clear}} = \min \|v_i - \text{closest obstacle}\|_2, \qquad d_{\text{avg,clear}} = \frac{\sum_{i=0}^{n} \|v_i - \text{closest obstacle}\|_2}{n}$$

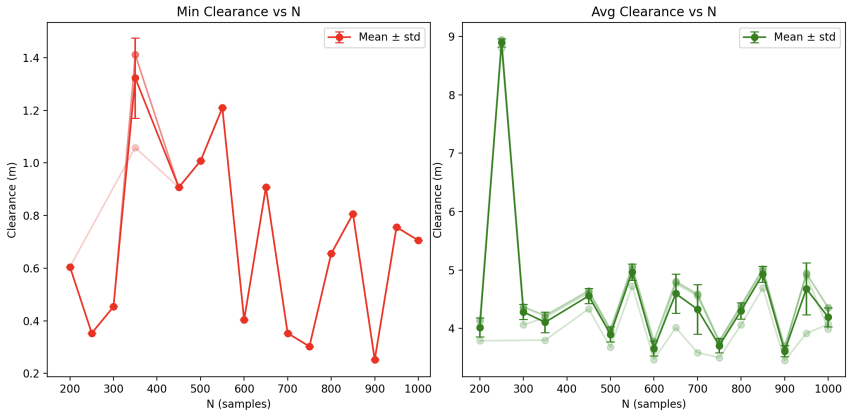

Figure 13: Clearance distances vs. n (peak at $n = 250$ is an outlier due to graph disconnection).

Clearance showed weak correlation with n. Minimum clearance remained above 0.2m, suggesting map inflation effectively prevents wall-hugging trajectories, no matter the graph size.

Heuristic Comparisons for the Online Planner

Though we used Dijkstra's in the final implementation of the online planner, we also compared three different path-search algorithms. Specifically, we fit three separate heuristics (0 = Dijkstra, Euclidean, Manhattan) to the A* algorithm across small and large graphs.

Small graph (n small): All heuristics performed similarly.

Table 1: A* Performance on Small Graph (ms)

Heuristic	**Mean**	**Std**	**Min**	**Max**
Euclidean	1939.95	967.74	264.41	3299.73
Manhattan	1923.35	881.85	212.31	3409.02
Dijkstra	1842.11	889.69	231.52	3525.85

Large graph ($n = 5000$): Euclidean and Dijkstra significantly outperformed Manhattan.

Table 2: A* Performance on Large Graph, $n = 5000$ (ms)

Heuristic	**Mean**	**Std**	**Min**	**Max**
Euclidean	32 243.05	939.04	31 147.14	33 351.88
Manhattan	40 634.90	5030.93	33 525.08	44 420.50
Dijkstra	32 517.54	2042.79	30 503.84	35 896.44

Euclidean A* performed nearly identically to Dijkstra, suggesting that heuristic computation overhead may offset any search reduction from A*. From the analysis of the parameters above we determined that the best model for our mission context in terms of efficiency, optimality, and safety is using a PRM of size $n = 250$ and Dijkstra's path-search algorithm.

Grid Search Based Planning: Effect of Step Size

Following the sampling-based approach, we evaluated our grid search approach based on the efficiency of the returned path. We define an efficient path to be one that is computed as quickly as possible while maintaining a cost as close as possible to the optimal cost. Continuing from the discussion of Single vs Multi-Step Exploration, we slowly raised the step size from 1, to determine the effects of such an increase on the efficiency of the paths returned. In Figure 14, we are able to see that increasing the step size only causes a slight increase in the returned path's distance compared to the optimal cost (shown by a step size of 1) regardless of how far the goal was placed. However, in terms of computation time, the cost to find the optimal path increased exponentially as the goal was placed farther away, whereas the cost of finding a slightly suboptimal path barely increased. From this analysis, we concluded that increasing the step-size had a positive effect on the efficiency of the returned path.

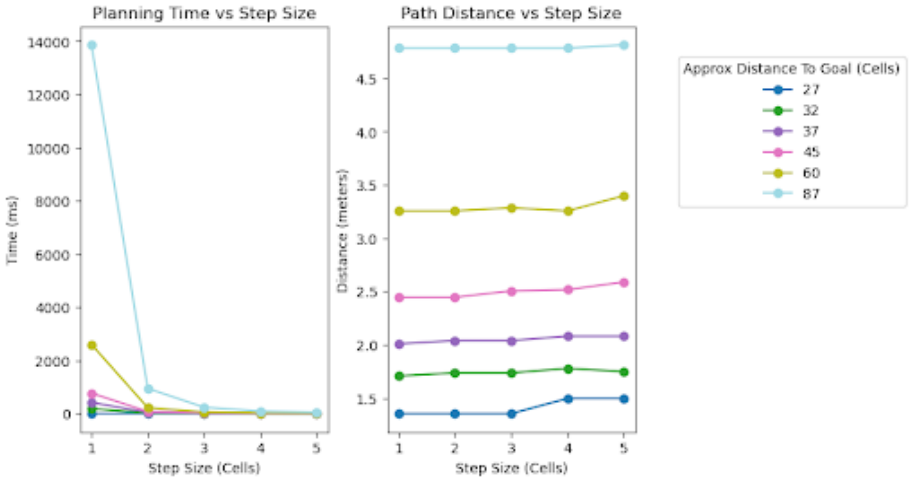

Figure 14: **Grid Search Planner** - Tradeoff between Planning Time and Path Distance as Step Size is increased. (Optimal path represented as step size of 1)

4.2.2 Path Planner Comparison

When deciding whether to use Sampling vs Grid Search for our path planner, we compared both based on the same **efficiency** metrics, computation time and path cost, and on the same **safety** metrics, average and minimum clearance from obstacles as the distance to the goal increased. For this comparison, we used Multi-Step Exploration with a step size of 10 for the Grid Search planner.

Efficiency
As shown in Figure 15, we were surprised to find that Grid-Search and Sampling were both able to find similar paths in terms of total distance, but Grid-Search was able to maintain a very low computation time (around 50 ms), whereas Sampling seemed to have a base computation time 8x higher (around 400 ms), that only increased as the goal was placed father away.

Safety
As shown in Figure 17, while Grid-Search and Sampling had similar minimum clearances to obstacles, Grid-Search had a tendency to "hug the wall" for long periods of time which is especially evident at higher distances from the goal where the planner has more opportunities to choose to save time by sticking close to the wall. This resulted in a path that was less safe, and that made sharp turns around corners as compared to the path returned by the Sampling-based planner, which tended to drive more in the middle of hallways, and make rounder turns.

Final Decision Ultimately, we decided to use the Sampling-based approach for our path planner as it tended to produce a safer path with a similar if not better cost than Grid Search. Even though it took longer to compute a path, since the order of magnitude for both planners was milliseconds, in real life and for our purposes, the differences in their computation times are barely noticeable.

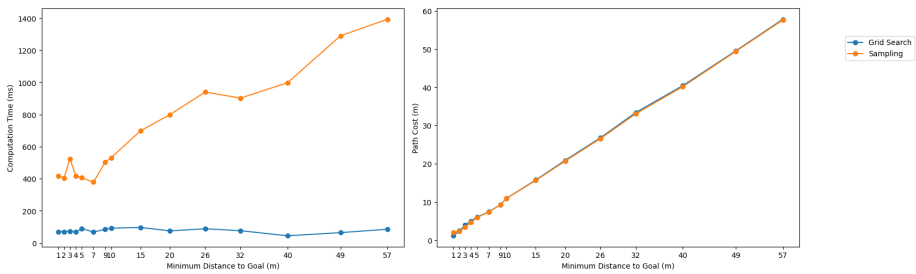

Figure 15: **Path Planner** - Efficiency Comparison

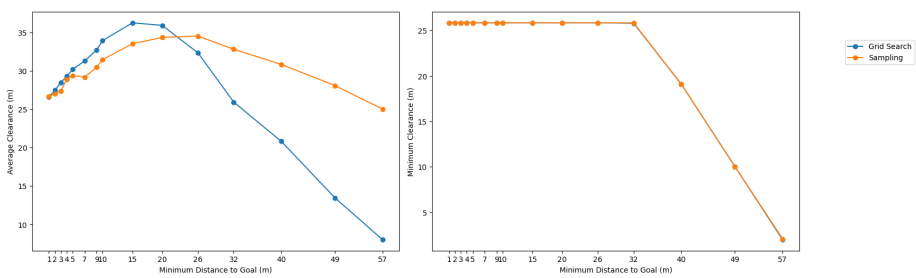

Figure 16: **Path Planner** - Safety Comparison

4.2.3 In Real-Life

We chose to use the sampling-based path planner in real life to compute paths since we were testing before fully optimizing of the grid planner. The provided trajectories maintained safe distances to obstacles and were traversable.

4.3 Path Following Results

4.3.1 Results in Simulation

In simulation, we have multiple trials with error less than a half car width for the entirety of the trajectory.

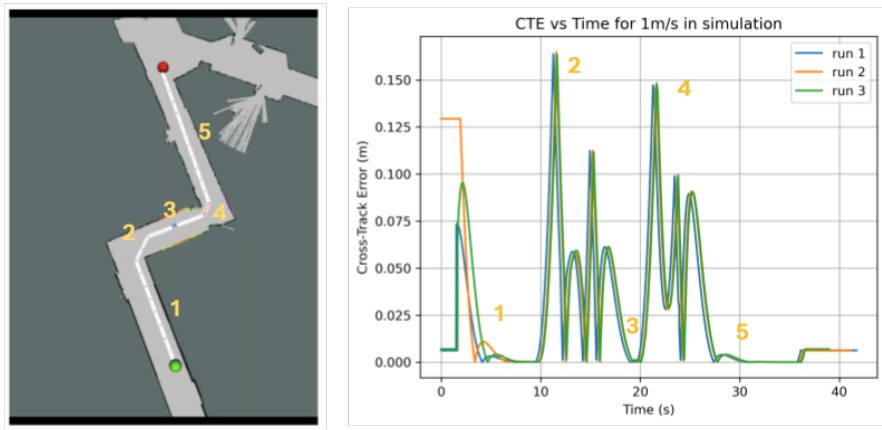

Figure 17: Low error in simulation with spikes in zones 2 and 4 indicating difficulty following trajectory around corners.

4.3.2 In Real-Life Results

Knowing that corners would be a difficult place to follow the trajectory, we incorporated localization with path following to test on the car. Overall CTE is higher which is to be expected as it takes into account error from the localization as well. At 1 m/s the spike in CTE at the corner is still present. It failed to both turn tightly enough and wide enough. To address the case of to wide of a turn, we decreased the speed when approaching a corner to allow it more precise maneuverability. To address the issue of turning too soon, we experimented with further discretizing the path (Fig. 19).

More intense discretization gives us more precision when navigating corners compared to the original, sparser trajectory. However, it introduces more oscillatory driving behavior in the longer straights. We can address this in one of two ways moving forward: either we can tune the lookahead distance to work for both straights and corners or we can intelligently discretize the trajectory only near corners (similarly to how we decrease the speed of the car near the corners).

5 Conclusion

Using LiDAR and odometry, we are able to use the Monte Carlo Localization method to estimate the position of the car in a 2D map both in simulation and in real life. With probabilistic road maps and Dijkstra's, the car is able to compute a trajectory efficiently. Finally, pure pursuit allows the car to follow

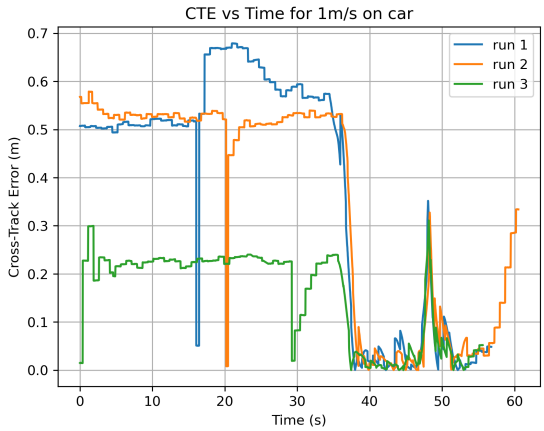

Figure 18: Success at low speeds with persistent spike around the corner.

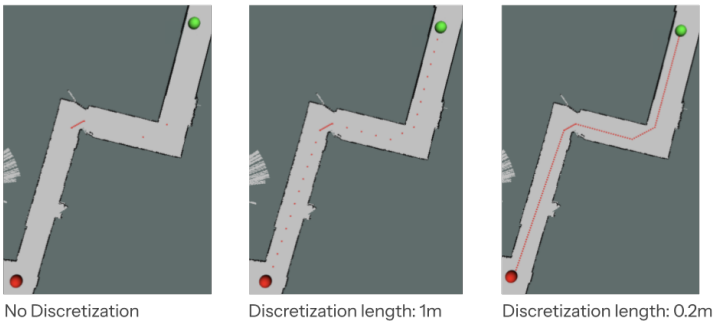

Figure 19: Effects of different discretization lengths on generated trajectories are most similar to the first corner of this given trajectory which is sparce to begin with.

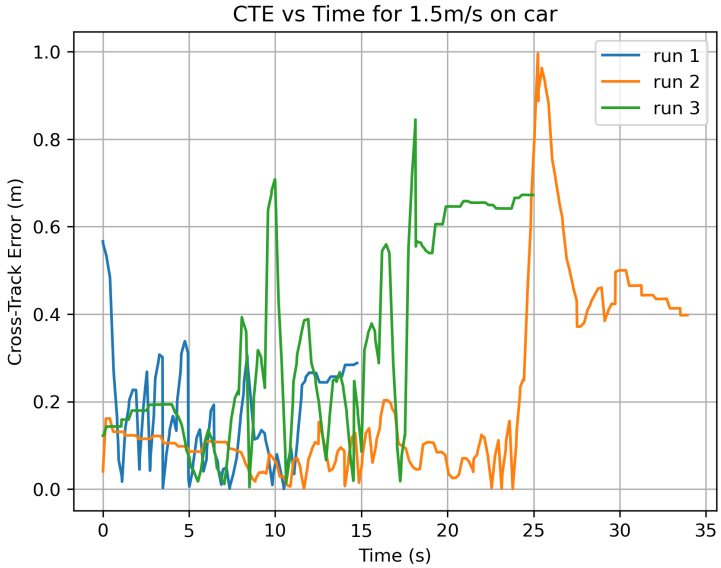

Figure 20: Oscillatory driving behavior as seen in interval $t \in [0, 10]$, corresponding to the straight part of the discretized trajectory.

this path smoothly. We refined our model by adjusting the noise we inject into the particle updates as well as the degree of our particle updates. Additionally, we compared different path finding approaches - sampling-based and grid-based - and found that sampling-based planning is most effective for our problem size. Through rigorous testing, we tuned the discretizations of our trajectory to match the needs of our pure pursuit algorithm.

Our current implementation does well in following the car's position through the map as it moves, but needs further improvement when navigating at higher speeds or through sharp turns. We will experiment further with the balance of the discretization length and lookahead distances to find what works in corners as well as straights. Given optimizations made to the grid based path planner, we may transition to using that as our preferred path planner. Refining these steps will be important as we head into the final challenge where we handle moving obstacles, corners, and timely planning.

6 Lessons Learned

Nina Garcia-Ruiz. Integrating several moving parts is difficult. It is important for everyone to understand how their part works and be able to communicate that so we can pinpoint sources of errors and areas to improve accuracy in the overall performance .

Brendon Murphy: Thoroughly test your models in simulation first before moving on to real-world development and testing. We tested our simulation for the feature-rich main hallway and moved on to the real-world without ensuring our particle filter was robust to changes in environment. This led to a lot of issues during testing that set us back and delayed completion of the lab. Testing everything in simulation first ensures implementation is sound, and makes it easier to fix changes rather than debug in real life

Irene Tang: Understanding how each part of the model works individually is helpful in debugging the whole system when something breaks. Additionally, comparing different algorithms using the same metrics is important to understand fundamental differences in performance.

Vandell Vatel: Having the same performance metrics when comparing differing methods helps to definitively tell which method is better and tradeoffs between them.